

ALMA MATER STUDIORUM - UNIVERSITÀ DI BOLOGNA

CAMPUS DI CESENA

CORSO DI LAUREA TRIENNALE IN INGEGNERIA E
SCIENZE INFORMATICHE

Relazione di Tirocinio Curricolare

Sviluppo di un'infrastruttura simulata in ambito sanitario

A.A. 2025-2026

Tirocinante:
Justin Carideo

Tutor Didattico:
Prof. Alessandro Ricci

Co-supervisore:
Ing. Angelo Croatti

Periodo di svolgimento: 02/03/2026 – 09/06/2026

Indice

1	Introduzione	2
2	Tecnologie	2
2.1	AnyLogic	3
2.2	Protocollo MQTT	3
2.3	Eclipse Paho	3
2.4	Eclipse Mosquitto	3
2.5	Docker e Docker-Compose	3
2.6	MQTT Explorer	3
3	Attività	4
3.1	Ubiquitous Language	4
3.1.1	Descrizione tramite UL	6
3.2	Simulazione e Protocollo ODD	6
3.2.1	Overview	7
3.2.2	Design Concepts	10
3.2.3	Details	10
3.3	Gestione del Broker MQTT	12
3.4	Testing tramite MQTT Explorer	12
4	Conclusioni	13

1 Introduzione

Nel contesto sanitario, l'efficacia della fase di soccorso extra-ospedaliera è strettamente legata all'adozione di metodologie rigorose e alla rapidità di intervento. Attualmente si utilizzano sistemi software in grado di supportare questo modus operandi, ma sorge l'esigenza di far evolvere tali strumenti sfruttando le tecnologie più moderne per ottimizzare l'intero processo di gestione delle emergenze.

Al momento, l'AUSL Romagna adotta un sistema mobile denominato **Trauma Tracker**, progettato per tracciare ogni manovra ed evento clinico-logistico che si verifica durante il soccorso di pazienti con trauma grave nella fase pre-ospedaliera. I soccorritori sul campo hanno a disposizione un dispositivo mobile equipaggiato con un'applicazione dedicata alla registrazione tempestiva delle procedure effettuate. Tale applicazione interagisce sia con un backend centralizzato, deputato al mantenimento dello storico dei dati, sia con trasmettitori radio basati su tecnologia *Beacon BLE*, in grado di comunicare direttamente con i dispositivi medici di tracciamento dei parametri vitali.

Questa analisi del sistema preesistente permette di delineare le linee guida fondamentali che il presente progetto deve ereditare ed estendere. In primo luogo, la tracciabilità degli eventi deve evolvere, attraverso l'integrazione di un ecosistema IoT, verso un monitoraggio puramente real-time; in secondo luogo, i dati raccolti devono consentire la creazione di un sistema di analisi a posteriori, abilitando una retrospettiva accurata delle dinamiche di soccorso. Di conseguenza, si rende necessario individuare un paradigma tecnologico capace di fondere queste due anime fondanti del progetto Trauma Tracker e la scelta più appropriata è stata individuata nei *Digital Twin*.

L'analisi dei requisiti e la modellazione del sistema partono dall'adozione della metodologia del **Domain-Driven Design** (DDD), un approccio strategico fondamentale per far sì che qualunque esperto del dominio clinico sia in grado di interpretare e comprendere ogni componente dell'architettura software creata, definendo un *Ubiquitous Language* condiviso. L'obiettivo del tirocinio si focalizza, pertanto, sulla progettazione di tale infrastruttura abilitante, ponendo le basi architetturali che verranno successivamente estese ed approfondite nel progetto di tesi.

La problematica principale nello sviluppo di un simile ecosistema risiede nell'impossibilità di validare il software in un contesto reale di emergenza, scenario che risulterebbe estremamente rischioso e controproducente. In tal senso, si è optato per la **simulazione** come tecnologia chiave per aggirare questi limiti operativi. L'integrazione di un motore di simulazione consente non solo di testare la reattività e la robustezza dell'architettura IoT in totale sicurezza, ma costituisce anche uno strumento vivo per l'apprendimento e la comprensione approfondita delle dinamiche complesse del dominio sanitario.

2 Tecnologie

Il raggiungimento degli obiettivi prefissati ha richiesto l'integrazione di diverse piattaforme software e protocolli di rete, volti a garantire il corretto disaccoppiamento dei componenti e la modularità dell'infrastruttura.

2.1 AnyLogic

AnyLogic è un motore di simulazione multifunzionale scritto in Java che supporta i principali paradigmi di modellazione: *Agent-Based*, *Discrete Event* e *System Dynamics*. All'interno del progetto, l'approccio orientato agli agenti è stato sfruttato per definire la granularità comportamentale e spaziale delle entità fisiche coinvolte (pazienti, equipaggi, veicoli di soccorso) all'interno di un ambiente continuo basato su mappe GIS (Geographic Information System) reali della regione Romagna.

2.2 Protocollo MQTT

Il protocollo MQTT (Message Queuing Telemetry Transport) è uno standard di messaggistica Pub/Sub (Publish/Subscribe) leggero, progettato per contesti IoT con risorse di rete limitate. La sua architettura basata su broker centralizzati garantisce il totale disaccoppiamento spaziale e temporale tra il Layer di Simulazione e gli applicativi esterni, consentendo di manipolare lo stato del modello attraverso l'*injection* di dati strutturati e asincroni in formato JSON.

2.3 Eclipse Paho

Eclipse Paho è una libreria open-source che fornisce implementazioni client affidabili per i protocolli MQTT. Nel contesto del progetto, le API Java di Paho sono state integrate direttamente all'interno delle classi custom e degli agenti di AnyLogic. Ciò ha permesso al modello simulativo di agire nativamente come un client MQTT in grado di sottoscrivere ai topic di comando e di pubblicare in tempo reale la telemetria geografica e i cambi di stato operativo.

2.4 Eclipse Mosquitto

Eclipse Mosquitto è un broker open-source che implementa le specifiche del protocollo MQTT. All'interno dell'architettura, Mosquitto funge da pilastro per lo scambio dei messaggi, prendendosi carico del routing dei payload JSON provenienti dai sistemi di iniezione dati o generati dalla simulazione stessa, garantendo latenze minime e alta affidabilità nella distribuzione degli eventi.

2.5 Docker e Docker-Compose

Docker è una piattaforma di containerizzazione che permette di isolare i servizi software all'interno di ambienti leggeri e riproducibili denominati *container*. Mediante l'utilizzo di Docker-Compose, è stata definita e orchestrata l'intera infrastruttura di rete locale (incluso il broker Mosquitto), azzerando le discrepanze di configurazione ambientale e isolando l'architettura per i test di deployment.

2.6 MQTT Explorer

MQTT Explorer è un client grafico completo per la visualizzazione e l'interazione con i broker MQTT. Caratterizzato da una struttura ad albero gerarchica per la

rappresentazione dei topic, lo strumento è stato impiegato come utility di controllo sia per monitorare i flussi di telemetria emessi da AnyLogic, sia per validare i meccanismi di iniezione dati attraverso la pubblicazione manuale di payload JSON di test.

3 Attività

Questa sezione illustra le fasi centrali di progettazione ingegneristica dell'infrastruttura.

3.1 Ubiquitous Language

In conformità con le linee guida del *Domain-Driven Design* (DDD), lo sviluppo dell'architettura è stato preceduto dalla formalizzazione del linguaggio ubiquitario (*Ubiquitous Language*). Questa attività mappa i concetti operativi e clinici della fase pre-ospedaliera (*PreH*) in termini tecnici condivisi tra sviluppatori ed esperti di dominio.

Tabella 1: Ubiquitous Language PreH

Termine	Definizione
Centrale Operativa	Unità operativa che abilita un Soccorritore e un Veicolo di Soccorso (Ambulanza o Elisoccorso) per dirigersi verso il luogo in cui prestare soccorso.
Soccorritore	Unità in grado di operare nel Scenario di Soccorso. Spesso è un volontario o dipendente che ha un addestramento al primo soccorso. Non può fare diagnosi o somministrare farmaci. Può guidare un mezzo (solitamente ambulanze e automediche) e fare manovre base di primo soccorso.
Infermiere	Unità in grado di fare le manovre di soccorso avanzate, sotto protocollo può somministrare farmaci. Collabora con il medico se presente sul mezzo o scenario. Di norma guida l'equipaggio della <i>India</i> (Ambulanza Avanzata).
Medico	Unità che ha la massima responsabilità sul target. Può fare diagnosi e somministrare farmaci. Decide la destinazione del Paziente e come gestire il soccorso nello Scenario di Soccorso.
Equipaggio	Insieme di professionisti (Soccorritori, Infermieri, Medici) che si trovano a bordo di un Veicolo di Soccorso.
Veicolo di Soccorso	Mezzo di trasporto (Ambulanza, Automedica o Elisoccorso) che viene utilizzato per muoversi e raggiungere lo Scenario di Soccorso ed eventualmente trasportare il Paziente.
Paziente	Il target che ha subito un trauma grave nel territorio e che necessita di essere soccorso e stabilizzato.
Scenario di Soccorso	Luogo geografico in cui si è verificato l'evento per cui è stato richiesto soccorso. All'interno operano gli Equipaggi.

Tabella 1

Termine	Definizione
Cinematica del Trauma	Studio delle forze che hanno colpito il Paziente a seguito di un incidente. Aiuta a comprendere la gravità delle lesioni interne anche in assenza di sintomi evidenti.
Parametri Vitali PreH	Insieme di dati clinici e fisiologici (frequenza cardiaca, pressione arteriosa, saturazione, GCS...) raccolti nello Scenario di Soccorso.
Evento PreH	Qualsiasi azione logistica o clinica (partenza del mezzo, arrivo sullo scenario, inizio di una procedura medica...) che avviene durante la gestione del soccorso.
Report PreH	Documento che raccoglie la cronistoria di tutti gli Eventi PreH e i Parametri Vitali PreH associati a una Missione.
Missione	Insieme di attività coordinate dalla Centrale Operativa, che partono dall'attivazione dell'Equipaggio fino all'Handover.
Handover	Atto formale e di responsabilità penale/medica con cui l'Equipaggio extra-ospedaliero affida il Paziente al personale del Pronto Soccorso.
Pronto Soccorso	Luogo ospedaliero attrezzato per la gestione delle emergenze, punto di arrivo di tutta la fase PreH.

Passiamo un utilizzo pratico di questi termini attraverso il *Domain Storytelling*:

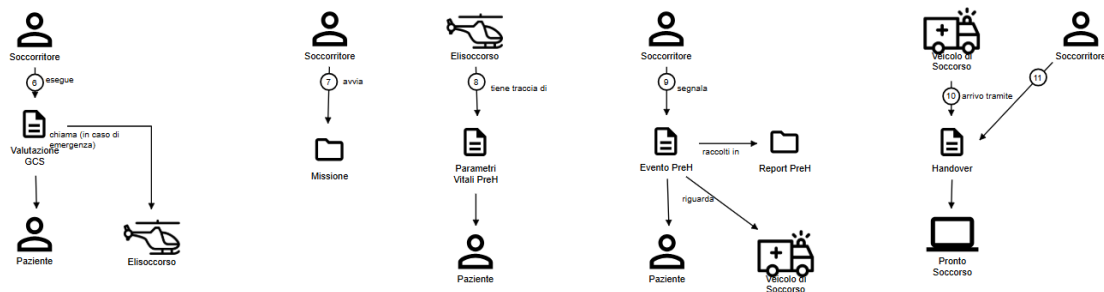


Figura 1: Domain Storytelling fase PreH

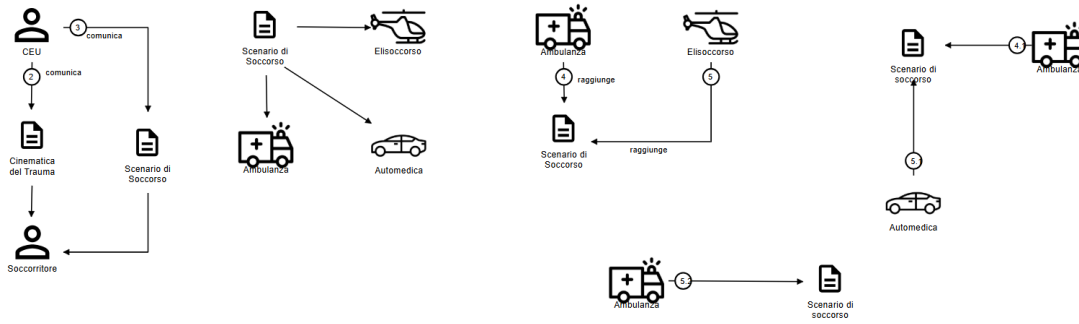


Figura 2: Domain Storytelling fase PreH - 2

3.1.1 Descrizione tramite UL

La **Missione** si attiva nel momento in cui un cittadino chiama il numero di emergenza (112/118) per contattare la **Centrale Operativa**. La Centrale Operativa segue dei protocolli per effettuare un'analisi preventiva sul **Paziente**, determinando il **Codice colore**. Dal Codice colore si determina la tempestività con la quale il Paziente deve essere soccorso. Una volta determinato il Codice colore, viene scelto un **Equipaggio** e i mezzi di soccorso.

Il veicolo raggiunge lo Scenario di Soccorso. L'Equipaggio mette in sicurezza l'area ed effettua una **Valutazione Clinica** basata su protocolli standard (che include il calcolo del GCS e di altri Parametri vitali). Questa valutazione conferma la gravità e determina quale sarà l'ospedale di destinazione.

La Centrale Operativa monitora attivamente i mezzi in movimento (es. tramite GPS e radio) e comunica in quale **Ospedale** muoversi. Durante l'intero svolgimento della missione, ogni azione significativa (arrivo del mezzo, avvio missione, ecc.) viene registrata come un **Evento**, dotato di un suo timestamp. Tutti gli Eventi, che riguardano direttamente il Paziente e il veicolo di soccorso coinvolto, vengono raccolti e storicizzati in un **Report**, un documento (fisico o digitale) che ha valenza sia medica che legale e mostra un resoconto della fase di tracciamento.

La Missione termina con la fase di **Handover**. Questo è il momento di intermezzo che segna il passaggio dalla fase extra-ospedaliera a quella intra-ospedaliera, il cui evento scatenante è l'arrivo fisico del veicolo e del Paziente al **Pronto Soccorso**. Il Pronto Soccorso rappresenta il luogo di destinazione finale di tutta la fase pre-ospedaliera.

3.2 Simulazione e Protocollo ODD

La simulazione è stata documentata attraverso il **Protocollo ODD** (Overview-Design Concepts-Details), utilizzato per descrivere modelli agent-based. Si divide in tre categorie concettuali:

- **Overview**: chi e cosa è presente nel modello.
- **Design Concepts**: principi alla base del comportamento degli agenti.
- **Details**: come è stato implementato il modello.

3.2.1 Overview

Lo scopo primario del modello è la riproduzione della catena di soccorso territoriale per valutare l'efficienza di allocazione delle risorse sanitarie, calcolare la distribuzione dei tempi di intervento e analizzare l'impatto clinico sul *Paziente* vittima di trauma grave.

I criteri di validazione del modello si basano sull'analisi di due metriche temporali critiche: il *Response Time* (dalla ricezione della chiamata all'arrivo del mezzo sullo scenario) e il *Total Pre-Hospital Time* (dalla chiamata all'avvio dell'fase di *Handover* in Pronto Soccorso).

Le entità che compongono il modello sono:

- **Ambulance / MedCar / MedHelicopter:** agenti fisici dotati di coordinate geografiche dinamiche e degli stati per tracciare lo stato operativo.
- **Patient:** entità dinamica generata a seguito di una segnalazione, caratterizzata da coordinate spaziali statiche di evento, un timestamp di attivazione e un codice di gravità clinica. In questo contesto l'agente simula i sensori che dovrebbero essere attaccati al paziente.
- **Main:** l'agente di contesto che modella il territorio e gli eventi che accadono. Al suo interno risiedono le popolazioni degli agenti e le funzioni di smistamento logistico dei messaggi.
- **CEU:** l'agente che si occupa delle scelte logistiche degli agenti. In particolare si occupa di smistare le segnalazioni e di indicare in quale ospedale deve essere portato il paziente.
- **Hospital / StationaryPoint:** sono agenti di contesto che vivono nel **Main** e servono per orientare gli altri agenti nell'ambiente.

Per avere più fedeltà con l'ambiente reale la simulazione ha scala temporale in **secondi** e una misura spaziale in **metri**.

Gli agenti con un comportamento hanno i seguenti statechart:

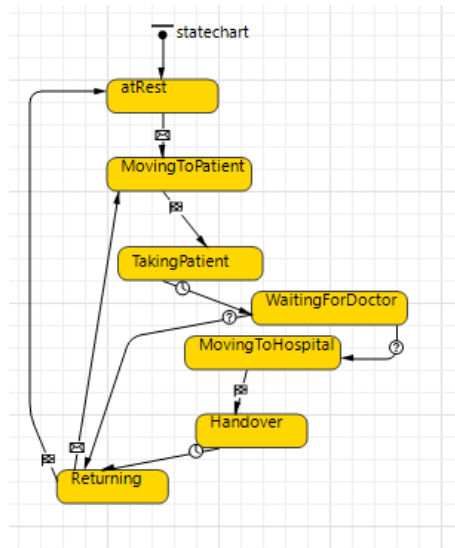


Figura 3: Statechart di Ambulance

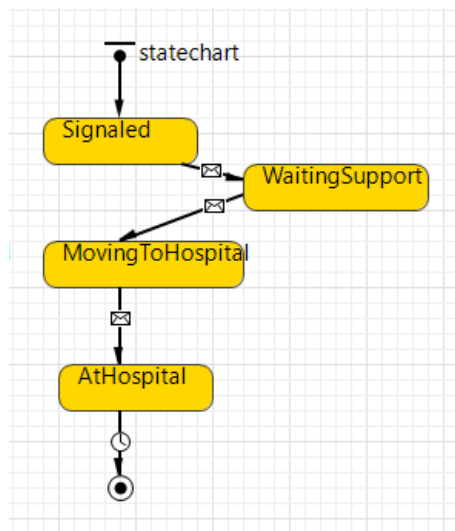


Figura 4: Statechart di Patient

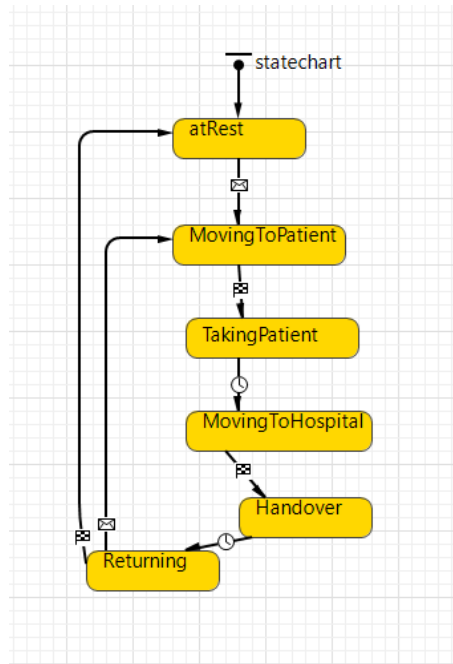


Figura 5: Statechart di MedHelicopter

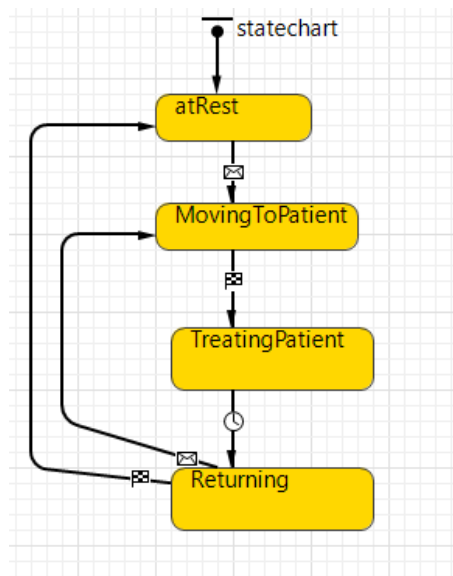


Figura 6: Statechart di MedCar

Esiste una parte di *Process Overview* che viene delineata in questa fase:

- **Signaling:** generazione di una segnalazione all'interno del Main
- **Dispatching:** CEU indica quali veicoli di soccorso sono coinvolti per il soccorso e dove è avvenuta l'emergenza
- **Caricamento:** i mezzi di soccorso arrivano sul luogo della segnalazione e caricano il paziente all'interno di uno dei veicoli.

- **Trasporto in ospedale:** il paziente viene trasportato in ospedale con gli strumenti più adatti alla gravità della situazione.
- **Handover:** il paziente viene trasportato all'interno dell'ospedale
- **Ritorno:** i mezzi ritornano negli opportuni punti stazionari da cui sono partiti.

3.2.2 Design Concepts

Le sezioni più importanti in questa fase sono:

- **Basic Principles:** il punto di innesco è la CEU che gestisce le segnalazioni e tramite una regola di dispatch seleziona i veicoli da mandare. Ogni altro veicolo ha un comportamento autonomo e comunica con gli altri in caso di necessità.
- **Emergence:** i fenomeni emergenti principali sono:
 - I ritardi non dipendono dal numero di mezzi ma dalla lontananza del target e conseguentemente dalla locazione dei punti stazionari.
 - Nonostante i soccorsi extraurbani il tempo di risposta medio tende a convergere.
 - Congestione ospedale in caso di chiamate simultanee.
- **Adaption:** la CEU implementa una regola di dispatch con fallback in caso di mancanza di risorse.
- **Interaction:** tramite *message passing* in caso di emergenza i veicoli coinvolti devono eseguire un *rendez-vous* per soccorrere il paziente. Per esempio se il codice è rosso e vengono coinvolte un'Ambulanza e un Elisoccorso, i veicoli dovranno incontrarsi nel luogo di emergenza e aspettare nel caso di mancanza di uno dei due.
- **Stochasticity:** in questa sezione vengono riportate tutte le variabili aleatorie, i.e. coordinate dell'emergenza, generazione delle segnalazioni, tempo di caricamento nel veicolo, tempo di handover e assegnazione codice gravità (nel caso la segnalazione sia simulata).

3.2.3 Details

Le due sezioni principali sono:

- **Initialization:** All'avvio della simulazione, l'agente Main inizializza l'ambiente geografico e configura le popolazioni di agenti.
- **Submodels:** Descrizione formale dell'implementazione degli agenti nel motore di simulazione.

La sezione *Submodels* non è troppo dissimile dalla parte di *Process Overview*, la particolarità sta nel fatto che in questi casi il protocollo ODD si dimostra *incrementale* e permette di prendere modelli già costruiti per costruirne di nuovi. Nella documentazione ho riportato tre modelli:

- **Submodel 1 - Generazione e Dispatching Geografico:** Le segnalazioni vengono generate dall'evento `signalEvent` seguendo una distribuzione di Poisson con parametro `lambdaSignals` (segnalazioni/ora). Le coordinate del `Patient` vengono generate tramite la funzione `randomPointInside()`, vincolata al poligono GIS della zona operativa. La CEU prende i mezzi più vicini al punto segnalato e usa una policy per decidere quali mezzi inviare, simulando l'intervista che viene fatta al momento della chiamata. Nel caso non ci sia nessun mezzo il paziente viene messo in una coda d'attesa e il primo mezzo che si libera verrà assegnato al primo nella coda.

```
handleSignal(patient):
    distance = patient.distanceTo(nearestHospital)
    if patient.severityCode == RED and distance > 10000:
        if nearestHelicopter != null:
            send(nearestHelicopter, ACTIVATE, patient)
        else:
            send(nearestMedCar, ACTIVATE, patient)
    if nearestAmbulance != null:
        send(nearestAmbulance, ACTIVATE, patient)
    else:
        addToQueue(patient)
```

- **Submodel 2 - Operazioni sul posto:** All'arrivo sul luogo della segnalazione, `Ambulance` transita nello stato `TakingPatient`. Se il `SeverityCode` del `Patient` è RED e un mezzo avanzato è stato attivato, l'Ambulanza transita nello stato di attesa `WaitingForDoctor`.

La sincronizzazione tra i veicoli è mediata dalla variabile booleana `medicalAssisted` dell'agente `Patient`:

- **Caso MedCar:** all'arrivo sul posto, `MedCar` transita nello stato `TreatingPatient` ed esegue le procedure di stabilizzazione. Al termine, imposta `medicalAssisted = true`. Questo agisce da trigger per `Ambulance`, che si sblocca dallo stato `WaitingForDoctor` e riparte verso `targetHospital`, mentre `MedCar` torna in stato `Returning`.
 - **Caso MedHelicopter:** avviene un rendez-vous tra `MedHelicopter` e `Ambulance` sul luogo dell'emergenza. È `MedHelicopter` a trasportare il `Patient` verso `targetHospital`, mentre `Ambulance` fornisce supporto sul posto prima di tornare alla base.
- **Submodel 3 - Esperimento di Ottimizzazione:** L'esperimento isola il modello dall'animazione visiva ed esegue iterazioni cicliche a durata fissa per

garantire consistenza statistica. Il parametro iterato è `operativeAmbulances`, con valori da `Min=4` a `Max=8` con `Step=1`. Al termine di ogni singola iterazione, l'esperimento estrae la media globale dei tempi di risposta dal dataset `responseTimeData` e la inietta nel grafico dei risultati. Questo modello può essere utile per individuare colli di bottiglia all'interno della simulazione.

3.3 Gestione del Broker MQTT

Per rendere la simulazione interagibile dall'esterno bisogna che esista un canale di comunicazione con essa. Per ottenere questo risultato ho usato **Eclipse Mosquitto** come *broker MQTT* e per abilitare la comunicazione con la simulazione ho optato per **Eclipse Paho**. L'agente `Main` della simulazione possiede una variabile `mqttClient` inizializzata nella sezione `On startup` che permette ad ogni agente della simulazione di interagire con un broker MQTT, connettendosi all'indirizzo di loopback `localhost` alla porta 1883 (`tcp://localhost:1883`). Per rendere l'infrastruttura più flessibile e portabile il broker è stato inserito in un container Docker, grazie al fatto che esiste un'immagine mantenuta su Docker Hub. Attraverso l'introduzione del container è possibile istanziare un'architettura a servizi al di sopra della simulazione. Per non andare oltre alle finalità del tirocinio il broker verrà utilizzato nella prossima fase per testare la reattività della simulazione.

3.4 Testing tramite MQTT Explorer

In questa sezione vengono riportati i test con MQTT Explorer:

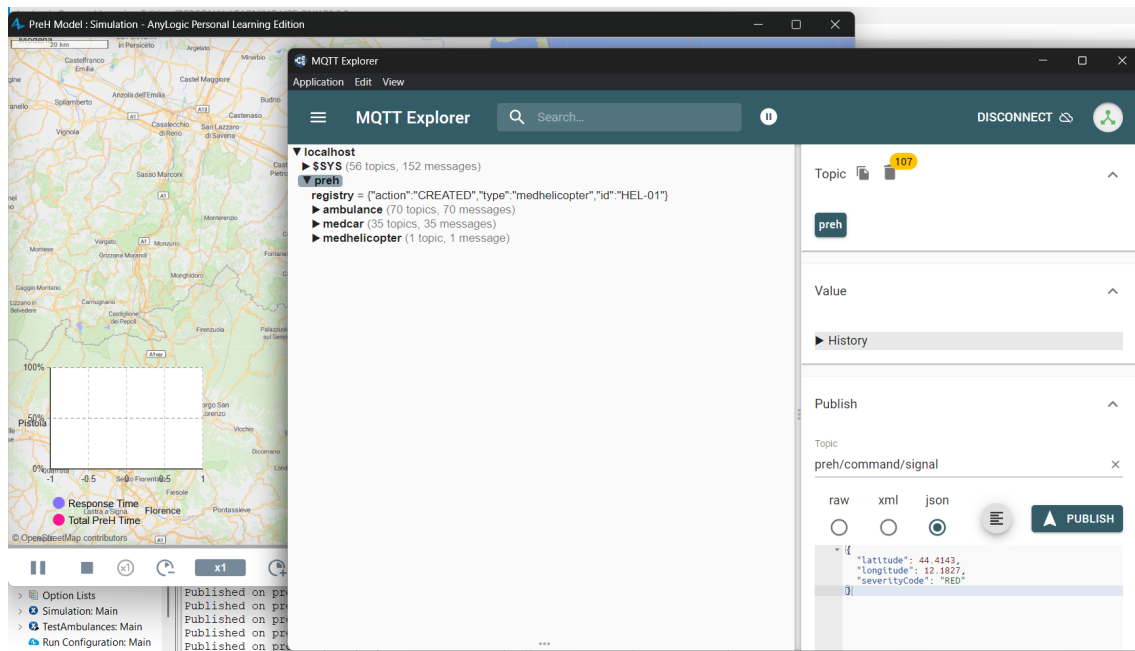


Figura 7: Istanziamento degli agenti tramite messaggi al broker e pubblicazione emergenza al topic `preh/command/signal`

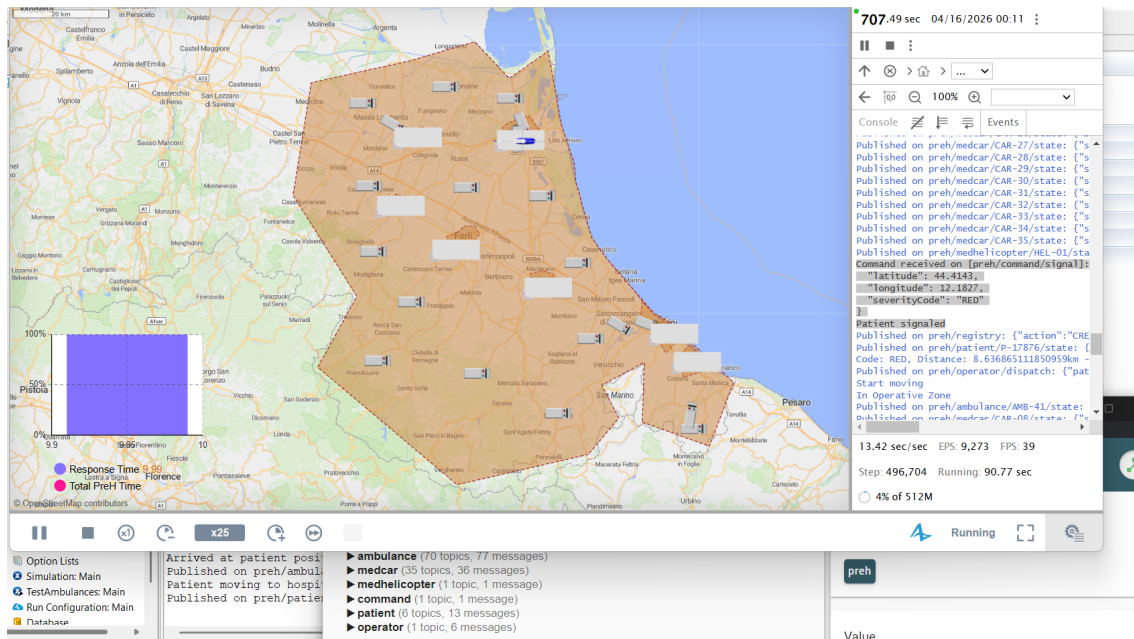


Figura 8: Reazione della simulazione alla pubblicazione del messaggio ricevuto dal broker

Come è possibile vedere nella Figura 3.4 e nella Figura 3.4 è possibile vedere che il meccanismo funziona. Questo risultato permette di rendere la simulazione ancora più manovrabile dall'esterno.

4 Conclusioni

Quanto svolto ha dei limiti, la simulazione in primis tenta di fare le veci della realtà e non è possibile modellare tutte le complessità presenti. Inoltre l'applicazione Any-Logic presenta alcuni difetti nel modellare alcuni aspetti reali tramite l'implementazione di cui dispone. Nonostante queste problematiche, il tirocinio mi ha permesso molto sul mondo della simulazione e di applicare quanto visto nel corso della laurea triennale, come il protocollo MQTT, Java e principi di ingegneria del software. Inoltre, il protocollo ODD ha permesso di rendere più fruibile anche la creazione e la relativa documentazione del modello creato, rendendolo più un lavoro di design che di effettiva implementazione. Il protocollo potrà essere valido in futuro anche per testare e verificare anche la comprensione del dominio e di utilizzare meccanismi di predizione che potrebbero basarsi su questo paradigma.

Riferimenti bibliografici

- [1] E. Evans, *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley, 2004.
- [2] V. Grimm, U. Berger, F. Bastiansen, S. Eliassen, V. Ginot, J. Giske, J. Goss-Custard, T. Grand, S. K. Heinz, G. Huse, et al., "A standard protocol for

- describing individual-based and agent-based models,” *Ecological Modelling*, 2006.
- [3] V. Grimm, U. Berger, D. L. DeAngelis, J. G. Polhill, J. Giske, and S. F. Railsback, “The ODD protocol: A review and first update,” *Ecological Modelling*, 2010.
- [4] R. Minerva, G. M. Lee, and N. Crespi, “Digital Twin in the IoT Context: A Survey on Technical Features, Scenarios, and Architectural Models,” *Proceedings of the IEEE*, 2020.
- [5] A. Ricci, A. Croatti, S. Mariani, S. Montagna, and M. Picone, “Web of Digital Twins,” *ACM Transactions on Internet Technology*, 2022.
- [6] A. Ricci, A. Croatti, and S. Montagna, “Pervasive and Connected Digital Twins—A Vision for Digital Health,” *Computing in Science & Engineering*, 2022.